Pen / IMU Timestamp Alignment

Overview: 00_Project_Overview

Bottom line

Pen and watch (IMU) clocks drift relative to each other. We correct the pen timestamps by choosing the time shift `delta` that **minimizes watch acceleration variance during pen strokes**.

Physical assumption: while the pen is on paper, the wrist holding the watch is comparatively still. A wrong `delta` places the stroke intervals on top of higher-variance arm motion, so the right `delta` shows up as a clear minimum.

Setup

Inputs:

- `imu_df` with columns `timestamp`, `AccX`, `AccY`, `AccZ` (sampled regularly).
- `pen_df` with columns `timestamp`, `StrokeID` (one or more samples per stroke).

Output:

- `delta_sec`: best shift to apply to pen timestamps.
- Diagnostics: minimum/mean/std of variance over the search grid, and the z-score of the minimum (`sigma_minimal_variance = (var_min - var_mean) / var_std`).
- Downstream: a boolean `in_stroke` column on the IMU after applying the shift.

Search grids in this repo (from `conf/base/parameters/merge.yml`):

dataset	coarse range / step	fine half-width / step
penstudy	`[-300, 300) s` at `1 s`	`+/- 20 s` at `0.01 s`
moleskine	`[-20, 20) s` at `0.5 s`	`+/- 5 s` at `0.01 s`

Math

Let IMU samples be $a_i = (x_i, y_i, z_i)$ at times t_i. The sample frequency is estimated from the median timestamp difference:

$$f_s = \frac{1}{\text{median}(t_i - t_{i-1})}$$

Choose a window of $W = \lfloor w \cdot f_s \rfloor$ samples, where w is `var_window_sec` (default `0.2 s`). Per-axis rolling variance:

$$s_{x,i} = \text{Var}\left(\{x_j : j \in \text{window}_i\}\right), \quad \text{idem for } y, z$$

Combine axes into a single normalized scalar variance feature:

$$v_i = \frac{\sqrt{s_{x,i}^2 + s_{y,i}^2 + s_{z,i}^2}}{\text{median}_j \|a_j\|_2}$$

The denominator (median acceleration norm, dominated by gravity) makes v_i comparable across subjects/devices. (Note: the implementation squares the per-axis variances before summing, i.e. it uses an L2 norm of the variance vector, not the sum of variances.)

Per stroke `s`, define the stroke interval $[\tau_s^{\text{start}}, \tau_s^{\text{end}}]$ on the pen clock. After candidate shift δ, the stroke mask on the IMU timeline is

$$m_i(\delta) = \begin{cases} 1, & \text{if } \exists s : t_i \in [\tau_s^{\text{start}} + \delta,\ \tau_s^{\text{end}} + \delta] \\ 0, & \text{otherwise} \end{cases}$$

The objective is the mean variance under the shifted stroke mask:

$$J(\delta) = \frac{1}{|\{i : m_i(\delta) = 1\}|} \sum_{i : m_i(\delta) = 1} v_i$$

We pick

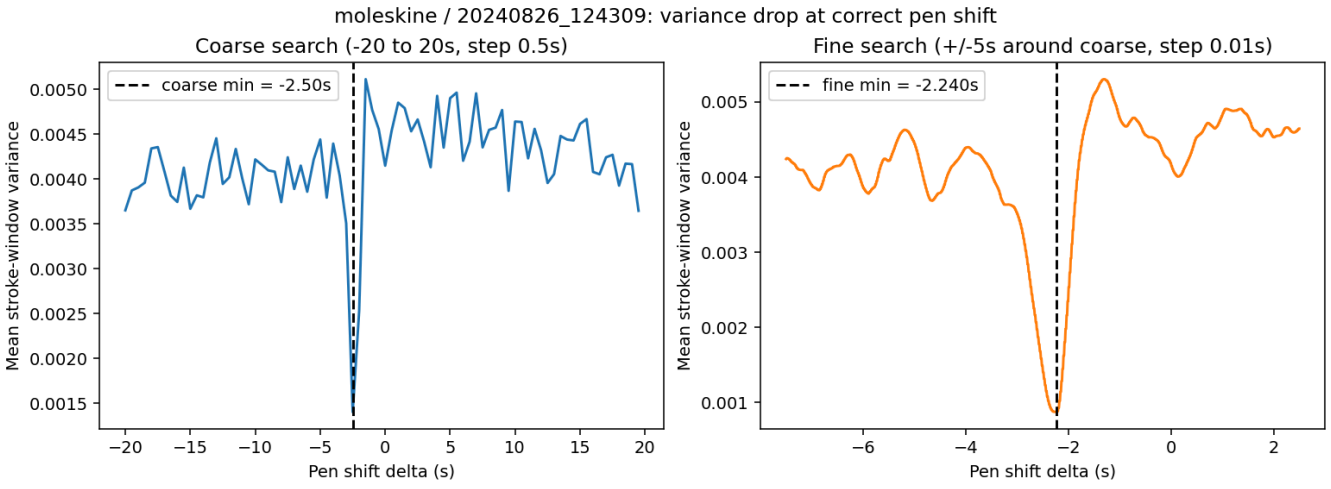
$$\delta^* = \arg\min_{\delta \in G} J(\delta)$$

evaluated on a coarse grid G first, then on a finer grid centered on the coarse minimum.

Variance drop plot

This figure is placed here to visually show the objective
in the math section above.

Example variance-vs-shift on Moleskine session `20240826_124309`
(coarse `[−20, 20) s` at `0.5 s`, fine `+/− 5 s` at `0.01 s`):



The clear, narrow well at `delta* ~ −2.24 s` is the alignment signal we minimize. Flat regions away from the well are the noise floor (mean variance over mostly non-stroke arm motion).

Reproduce with:

```
import matplotlib.pyplot as plt

minima, var_series = pen_match(
    imu_df,
    pen_df,
    start_delta_sec=-20.0,
    end_delta_sec=20.0,
```

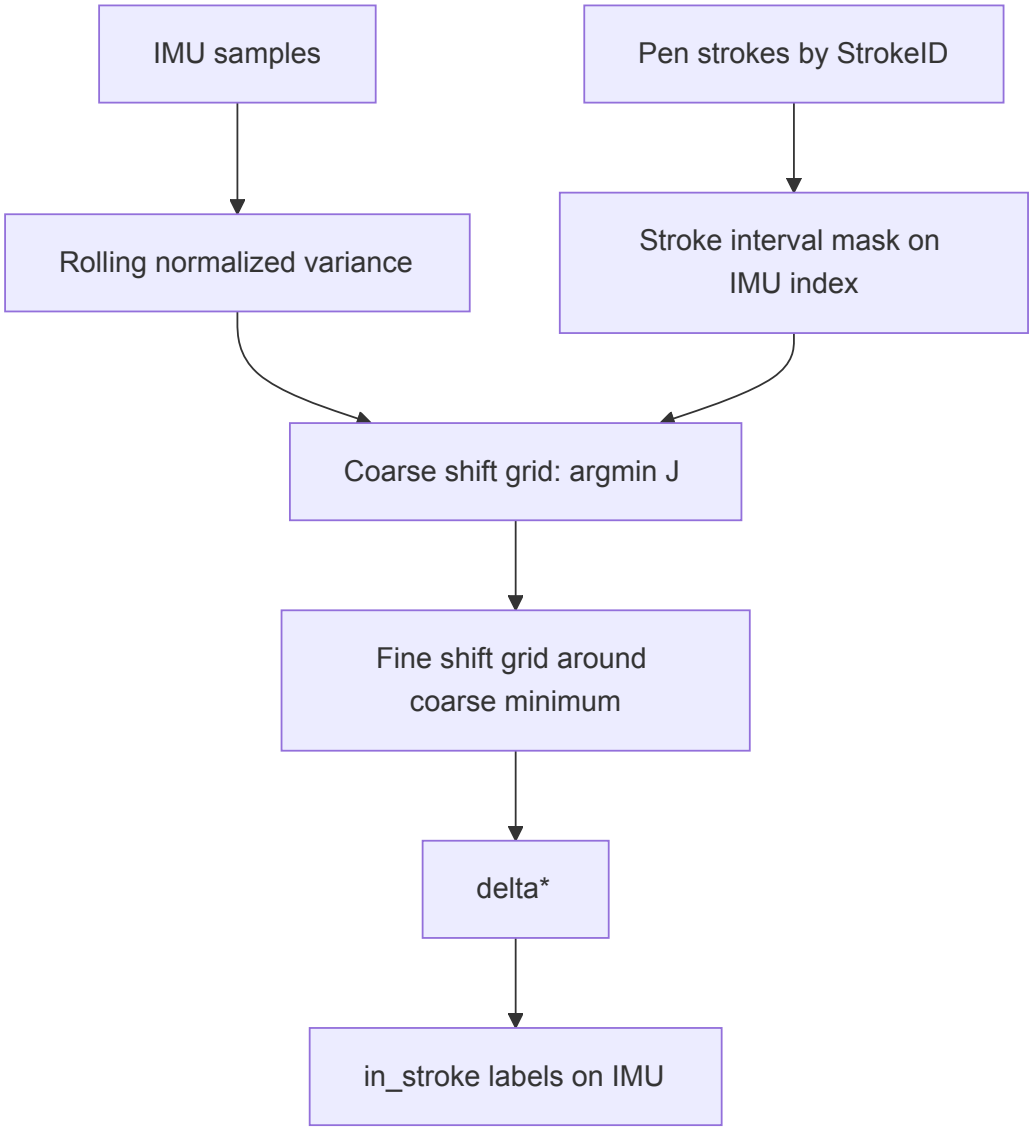
```
    step_sec=0.5,
)

ax = var_series.plot(
    xlabel="Pen shift delta (s)",
    ylabel="Mean stroke-window variance",
)
ax.axvline(minima[0], color="black", linestyle="--",
           label=f"best delta = {minima[0]:.2f} s")
ax.legend()
plt.show()
```

The static plot above is generated by `scripts/generate_pen_imu_alignment_plot.py`.

Algorithm

1. Estimate `f_s` from median sample interval.
2. Build the rolling variance vector `v` per IMU sample (window `var_window_sec`).
3. Aggregate pen samples to stroke intervals by `StrokeID` (min/max timestamp).
4. Build a stroke mask `m` on the IMU index (no shift yet).
5. For each `delta` in the coarse grid: shift `m` by `round(delta * f_s)` samples and take the mean of `v` under the shifted mask. The shift with the smallest mean is the coarse winner.
6. Repeat on a fine grid centered on the coarse minimum.
7. Apply `delta*` to pen timestamps and label IMU rows with `in_stroke`.



Reference implementation

The full function lives in `src/focuswatchalgok/pipelines/imu_pen_merge/merge_nodes.py`.
The core of `pen_match` is small:

```
def pen_match(
    imu_df,
    pen_df,
    start_delta_sec,
    end_delta_sec,
    step_sec,
    var_window_sec=0.2,
    acc_cols=["AccX", "AccY", "AccZ"],
):
    """Find time shift between IMU and pen stroke intervals.

    Builds a normalized rolling acceleration-variance
    signal on the IMU, masks samples that fall inside
    pen stroke time ranges, then evaluates mean
    variance over a grid of integer-sample shifts.
    The shift with lowest mean variance is returned.
```

```
    Args:
        imu_df: IMU samples in time order. Must include:
            - ``timestamp``: datetime-like, used to infer
              sample rate and for masking.
            - Columns named in ``acc_cols`` (default
              ``AccX``, ``AccY``, ``AccZ``):
              numeric accelerometer values.
        pen_df: Pen stroke samples. Must include:
            - ``StrokeID``: identifier to group strokes.
            - ``timestamp``: datetime-like; per-stroke
              min/max define the in-stroke interval on
              the IMU timeline (before applying shift
              search).
        start_delta_sec: Start of shift grid (seconds),
            passed to ``numpy.arange``.
        end_delta_sec: End of shift grid (exclusive),
            passed to ``numpy.arange``.
        step_sec: Step size (seconds) between candidate shifts.
        var_window_sec: Rolling window length (seconds)
            for variance on ``acc_cols``.
        acc_cols: Subset of columns in ``imu_df`` used
            for the variance feature.

    Returns:
        Tuple of ``(minima, var_series)`` where ``minima`` is
        ``(best_delta_sec, min_mean_variance)`` and
        ``var_series`` is a ``pd.Series`` of mean
        variance indexed by candidate ``delta_sec``.
    """
    freq_hz = (
        1 / imu_df["timestamp"].diff().median().total_seconds()
    )


    # calculate variance vector
    var_window_samples = int(var_window_sec * freq_hz)
    g_norm = (
        (imu_df[acc_cols].astype(np.float32) ** 2)
        .sum(axis=1)
        .apply(np.sqrt)
        .median()
    )
    var_vec = (
        imu_df[acc_cols]
        .astype(np.float32)
        .rolling(
            window=var_window_samples,
            center=True,
        )
        .var()
    )
    var_vec = (
        (var_vec ** 2).sum(axis=1).apply(np.sqrt)
        / g_norm
    )
```

```
    # shift to align with pen data
    var_vec = var_vec.shift(-var_window_samples)

    # Group pen data by StrokeID and calculate
    # start/end timestamps for each stroke.
    stroke_df = pen_df.groupby('StrokeID').agg(
        start=('timestamp', 'min'),
        end=('timestamp', 'max')
    ).reset_index()

    # generate mask vector
    mask_vec = pd.Series(False, index=imu_df.index)

    for _, row in stroke_df.iterrows():
        mask = (
            (imu_df["timestamp"] >= row["start"])
            & (imu_df["timestamp"] <= row["end"])
        )
        mask_vec[mask] = True

    deltas_sec = np.arange(start_delta_sec, end_delta_sec, step_sec)
    deltas_samples = np.int32(np.round(deltas_sec * freq_hz))

    var_values = [
        var_vec[
            mask_vec.shift(
                delta,
                fill_value=False,
            )
        ].mean()
        for delta in deltas_samples
    ]
    var_series = pd.Series(var_values, index=deltas_sec)

    # Find the index of the minimal value in var_values
    min_index = np.argmin(var_values)

    # Get the corresponding delta_sec value
    minima = (
        deltas_sec[min_index],
        var_values[min_index],
    )

    return minima, var_series
```

The wrapper that runs coarse-then-fine and returns diagnostics:

```
def match_pen_data_node(
    imu_df: pd.DataFrame,
    pen_df: pd.DataFrame,
    pen_match_params: dict,
) -> dict:
    """Match pen data with IMU data for a single subject.
```

```
    Returns:
        dict: Match statistics (plain floats for YAML persistence).
    """
    cs = pen_match_params["coarse_start_delta_sec"]
    ce = pen_match_params["coarse_end_delta_sec"]
    cstep = pen_match_params["coarse_step_sec"]
    fh = pen_match_params["fine_half_width_sec"]
    fstep = pen_match_params["fine_step_sec"]

    minima, _ = pen_match(imu_df, pen_df, cs, ce, cstep)
    minima2, var2 = pen_match(
        imu_df, pen_df, minima[0] - fh, minima[0] + fh, fstep
    )

    # Native floats so YAMLDataset round-trips with yaml.safe_load (no numpy tags).
    mean_v = float(var2.mean())
    std_v = float(var2.std())
    return {
        'delta_sec': float(minima2[0]),
        'minimal_variance': float(minima2[1]),
        'average_variance': mean_v,
        'stddev_variance': std_v,
        'sigma_minimal_variance': float((minima2[1] - mean_v) / std_v),
    }
```

Standalone pseudocode

For collaborators who do not use Kedro, this
is the same algorithm in ~30 lines:

```
import numpy as np
import pandas as pd

def pen_match(
    imu_df,
    pen_df,
    start_delta_sec,
    end_delta_sec,
    step_sec,
    var_window_sec=0.2,
    acc_cols=("AccX", "AccY", "AccZ"),
):
    fs = 1 / imu_df["timestamp"].diff().median().total_seconds()

    W = int(var_window_sec * fs)
    acc = imu_df[list(acc_cols)].astype(np.float32)
    g_norm = np.sqrt((acc ** 2).sum(axis=1)).median()
    var_vec = acc.rolling(window=W, center=True).var()
    var_vec = np.sqrt((var_vec ** 2).sum(axis=1)) / g_norm
    var_vec = var_vec.shift(-W)

    strokes = pen_df.groupby("StrokeID")["timestamp"].agg(["min", "max"])
    mask = pd.Series(False, index=imu_df.index)
    for _, row in strokes.iterrows():
```

```
        mask |= (
            (imu_df["timestamp"] >= row["min"])
            & (imu_df["timestamp"] <= row["max"])
        )

    deltas_sec = np.arange(start_delta_sec, end_delta_sec, step_sec)
    deltas_samples = np.round(deltas_sec * fs).astype(int)
    var_values = [
        var_vec[mask.shift(d, fill_value=False)].mean()
        for d in deltas_samples
    ]

    var_series = pd.Series(var_values, index=deltas_sec)
    best = var_series.idxmin()
    return (best, var_series.loc[best]), var_series
```

Interpretation and caveats

- The objective aligns clocks; it does not detect writing on its own. It assumes pen-derived stroke intervals already exist and that the pen clock has a roughly constant offset over the session.
- Strokes must occur while the watch is comparatively still. Tasks with large wrist motion during writing (for example holding or flipping the page mid-stroke) flatten the well.
- The grid must contain the true offset. If the well is at the edge of the search range, expand the range.
- Diagnostics: a *small* `minimal_variance` and a *strongly negative* `sigma_minimal_variance` (z-score of the minimum vs the search-grid distribution) indicate confidence. A flat curve means weak alignment evidence and `delta*` should not be trusted.
- Sample-rate regularity matters. `f_s` is estimated from `median(diff(timestamp))`; jittery streams should be resampled first.
- The two-stage coarse/fine search is a practical optimization, not a different objective: the coarse step must be smaller than the basin width.